

Eclat 对 Apriori 算法的改进

汇报人：常根

国际关系学院 · 网络空间安全学院
数据挖掘算法与应用

代码与数据：

`github.com/timegoer/eclat-vs-apriori`

2026 年 10 月 8 日

目录

1. 背景与问题定义
2. Apriori 的瓶颈
3. Eclat 的改进
4. 实验设计
5. 实验结果与分析
6. 总结
7. 参考文献

研究问题

Apriori 在每一层都需重新扫描数据库并生成大量候选集；Eclat 能否通过**垂直数据格式与集合交运算**规避这两项开销？

1. **瓶颈定位**：Apriori 的开销来源；
2. **改进机制**：Eclat 如何用垂直格式与集合交替代重复扫描；
3. **实验验证**：在公开数据集上的复现，验证改进的实际效果。

频繁项集挖掘：问题定义

- 设 $\mathcal{D}$ 为事务数据库，每条事务是一个项的子集；项集 X 的支持度 $\text{sup}(X)$ 定义为包含 X 的事务条数。
- 给定最小支持度阈值 min_sup ，频繁项集挖掘的目标是找出所有满足 $\text{sup}(X) \geq \text{min_sup}$ 的项集 X 。
- 该问题是关联规则挖掘 ($X \Rightarrow Y$) 的基础，应用于购物篮分析、安全告警日志分析与入侵检测规则生成等场景。

直观理解

若 1000 笔交易中有 30 笔同时包含“啤酒”与“尿布”，则该 2-项集的支持度为 30；支持度越高，说明这一组合出现得越普遍、越值得关注。

核心困难

含 $|I|$ 个项的项集空间规模为 $2^{|I|}$ ，逐一枚举不可行，必须依靠剪枝缩减搜索空间。

Apriori 的核心机制（一）：反单调性

向下封闭性（反单调性）

若项集 X 频繁，则其任意子集均频繁；等价地，若 X 的某个子集非频繁，则 X 及其所有超集均非频繁，可整枝剪除。

直观理解

一个组合要在数据中出现，它包含的每个子组合都必须先出现；因此若某个子组合本身出现得不够多，再向其中加入新的项，只会使出现次数进一步减少。

- 例：若 $\{A, B\}$ 仅出现 2 次，而 $\text{min_sup} = 5$ ，则 $\{A, B, C\}$ 、 $\{A, B, D\}$ 等一切超集的出现次数至多为 2，无须计算即可判定为非频繁。

意义

反单调性是所有频繁项集挖掘算法共享的剪枝基础：它将“逐个验证”转化为“用短项集的结果排除长项集”。

Apriori 的核心机制（二）：逐层搜索及其开销

1. **计数**：扫描数据库，统计各单项的支持度，得到频繁 1-项集 L_1 ；
2. **生成**：由 L_{k-1} 自连接生成 k -项候选集 C_k ，并依据反单调性剪枝；
3. **验证**：**再次扫描数据库**，统计 C_k 中各候选的支持度，得到 L_k ；
4. **迭代**：令 $k \leftarrow k + 1$ ，重复第 2-3 步，直至不再产生新的频繁项集。

直观理解

该过程类似于“搭积木”：先筛出合格的单件，再由合格的单件拼出两件组合、三件组合……而每搭一层，都必须把全部原始数据重新过一遍。

关键观察

k 每增加 1，数据库即被完整扫描一次；这是 Apriori 的第一项主要开销。

瓶颈之一：数据库的多次全量扫描

- 若最长频繁项集的长度为 k_{max} ，数据库至少被完整扫描 k_{max} 次；
- 当数据规模超出内存容量、或存储于磁盘与数据库系统时，扫描本身即成为主导开销；
- 每次扫描还须对每条事务执行**候选子集匹配**，比较次数约为 $|D| \times |C_k|$ 。

直观理解

若最长频繁项集含 10 个项，则意味着数据库要被重复读取 10 遍；事务数 $|D|$ 越大（例如 10 万条），每一遍的代价越高。

瓶颈之二：候选集膨胀与无效计数

候选集规模失控

候选集 C_k 的规模可能**远大于**实际频繁的项集数 L_k ，且随 k 组合式增长；每个候选都需在完整数据库上匹配计数，其中大量计算最终只用于判定“非频繁”。

开销类型	具体表现
输入/输出开销	每层重扫全库， k_{max} 次以上
计算开销	$ D \times C_k $ 次子集匹配
空间开销	维护规模庞大的候选集 C_k

表: Apriori 的三类开销，亦即 Eclat 的三个改进切入点。

直观理解

其本质是“先枚举所有可能的组合、再回到数据中逐个核对”；而绝大多数候选最终被判定为非频繁，相应计算均为无效开销。

改进一：数据布局的转置——由水平格式到垂直格式

水平格式 (Apriori 的视角)

事务编号 $\rightarrow$ 项集

- T1: {A, B, C}
- T2: {A, C}
- T3: {A, B}

垂直格式 (Eclat 的视角)

项 $\rightarrow$ *tidset* (含该项的事务编号集合)

- A: {1, 2, 3}
- B: {1, 3}
- C: {1, 2}

转置的代价与收益

垂直格式可在**一次完整扫描**数据库的过程中构建完成；此后所有的支持度计算都只涉及 *tidset* 之间的集合运算，不再访问原始事务数据。

改进一（续）：垂直格式为何能够常驻内存

前提：转置本身并不减少数据总量

两种格式所记录的都是全部（项，事务）出现关系，条目总数相同；真正的差别在于**后续计算需要访问哪些数据**。

1. **计算与数据规模解耦**：支持度仅由相关项的 tidset 交集决定，无须再次遍历全库，从而摆脱 Apriori 中 $|D| \times |C_k|$ 量级的重复扫描。
2. **可裁剪**：仅保留 $\text{sup} \geq \text{min_sup}$ 的项，非频繁项连同其 tidset 一并丢弃；阈值越高，保留量越小。
3. **可压缩**：tidset 可用位图（bit-vector）表示，每个事务占 1 个比特，集合交退化为按位与；故垂直数据库等价于一份倒排索引，其内存占用可控制在可接受规模。

直观理解

这相当于预先为数据集建立一个“每件商品出现在哪些小票上”的索引；此后再查询任意商品组合，只需取出索引中相应的行做比对，不必反复翻检全部小票。

改进二：支持度即集合交的大小

核心恒等式

$\text{sup}(X \cup Y) = |t(X) \cap t(Y)|$ ，其中 $t(\cdot)$ 表示项集的 tidset。

- 例： $t(A) = \{1, 2, 3\}$ ， $t(C) = \{1, 2\}$ ， 则 $\text{sup}(\{A, C\}) = |\{1, 2\}| = 2$ ；
- 求 k -项集的支持度无须再扫描数据库，只需对两个 $k-1$ 项集的 tidset 做一次交集；
- 剪枝准则随之改变：若 $|t(X) \cap t(Y)| < \text{min_sup}$ ，则任何包含 $X \cup Y$ 的超集均不可能频繁，可整枝剪除。

直观理解

两个项的 tidset 交集，正是“同时购买这两件商品”的交易集合，其元素个数即为该组合的支持度；求支持度因而退化为一次集合求交。

改进三：基于前缀等价类的深度优先搜索

- 将频繁项按固定顺序排列；固定前缀 P 后，仅在其后继续扩展（前缀等价类），从而保证每个项集恰好被枚举一次；
- 采用 DFS 递归：得到一个频繁项集后，立即以其 tidset 与后续项求交，直接产出下一层的频繁项集；
- 由此不再需要“生成候选一再验证”的循环，每一步都建立在已通过支持度检验的 tidset 之上，天然只产生频繁项集。

直观理解

Apriori 相当于逐层铺开、每层回到数据库核对一次；Eclat 则沿一条路径持续深入，直至无法继续再行回溯，整个过程均在内存中完成。

Apriori 与 Eclat 的机制对比

维度	Apriori	Eclat
数据格式	水平 (事务 $\rightarrow$ 项)	垂直 (项 $\rightarrow$ 事务编号)
扫描数据库	k_{max} 次以上	1 次 (预处理后即止)
搜索策略	逐层 BFS + 候选生成	前缀等价类 DFS
支持度计算	候选与事务的子集匹配	tidset 交集的大小
主要内存开销	候选集 C_k	搜索树上的全部 tidset
主要短板	大 $ D $ 上的重复扫描与候选膨胀	需在内存中维护垂直索引 (空间换时间)

实验设计

数据集（均为公开数据集）

数据集	事务数	项数	来源
mushroom	8,124	119	FIMI
T10I4D100K	100,000	870	FIMI

- **实现**：两种算法均为纯 Python 多线程实现，共用同一套数据加载代码，使用完全相同的 `min_sup`；
- **自变量**：仅支持度阈值，逐档降低；
- **度量**：运行时间（重复运行取最小值）、峰值内存、频繁项集数量；
- **正确性前置校验**：两算法输出的频繁项集必须**完全一致**，再比较性能；
- **计时范围**：仅算法本体，不含文件读取与结果输出。
- **可复现性**：全部代码、数据与实验结果已开源，见 github.com/timegoer/eclat-vs-apriori。

实验结果（一）：运行时间与加速比

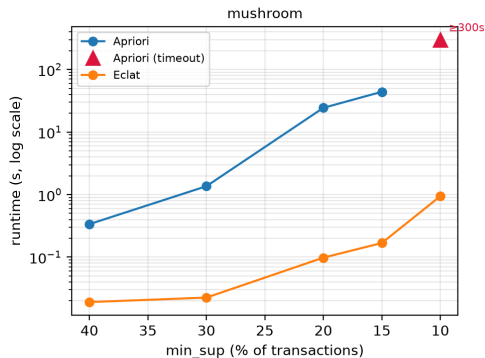
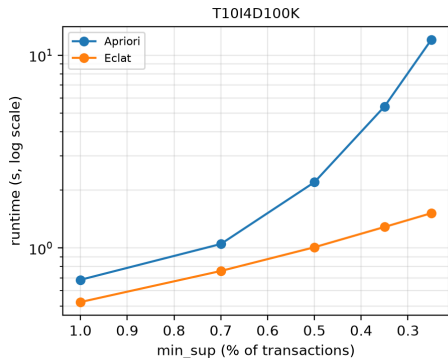
数据集 (min_sup)	频繁项集数	Apriori (s)	Eclat (s)	加速比
mushroom (30%)	2,735	1.36	0.02	60×
mushroom (20%)	53,583	24.35	0.10	248×
mushroom (10%)	574,513	超时 (> 300)	0.94	≥ 319×
T10I4D100K (1.0%)	385	0.68	0.52	1.31×
T10I4D100K (0.5%)	1,073	2.19	1.01	2.17×
T10I4D100K (0.25%)	7,703	12.04	1.51	7.95×

表：两个公开数据集上的实测结果。两算法输出的频繁项集完全一致（已校验）。Eclat 为冷启动口径，含一次性垂直索引构建（mushroom 约 0.02 s，T10I4D100K 约 0.12 s）。

要点

在**所有测试配置**下 Eclat 都快于 Apriori，且**支持度越低、优势越大**；mushroom 在 10% 支持度时，Apriori 运行 300 s 仍未完成，而 Eclat 仅用 0.94 s 即挖出全部 57 万个频繁项集。

实验结果（二）：运行时间随支持度的变化



- 两个数据集上，Eclat 的曲线始终位于 Apriori 下方；纵轴为对数刻度；
- 随支持度降低，Apriori 需处理的层数与候选集同步增加，耗时上升更快，两条曲线的间距**持续扩大**（mushroom 上 Apriori 最终超出 300s 时限）。

总结

1. **瓶颈**: Apriori 采用“逐层扫描 + 候选验证”的范式，每一层都要重新完整扫描数据库，并生成与计数大量候选集；
2. **改进**: Eclat 将数据库一次性转置为垂直格式，以集合交替代“扫描计数” ($\text{sup}(X \cup Y) = |t(X) \cap t(Y)|$)，并以基于前缀等价类的深度优先搜索替代逐层候选生成；
3. **验证**: 在 2 个公开数据集上扫描支持度实测，Eclat 在**全部配置**下均快于 Apriori，且加速比随支持度降低而单调扩大（最高超过两个数量级）；在两者结果完全一致的前提下，改进得到验证。

结论

Eclat 通过改变数据布局与支持度计算方式，确实有效优化了 Apriori 的性能瓶颈。

局限与展望

- 实验中 Apriori 为朴素实现，后续可采用 SPMF、efficient-apriori 等成熟实现作为更公平的基线；
- 本次仅比较两个算法，未评测 Eclat 的其他变体；
- 后续工作：FP-Growth 的树结构，以及垂直格式的并行与分布式实现。

参考文献 I

- [1] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Washington, DC, USA, 1993, pp. 207–216.
- [2] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. on Very Large Data Bases (VLDB)*, Santiago, Chile, 1994, pp. 487–499.
- [3] M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, "New algorithms for fast discovery of association rules," in *Proc. 3rd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, Newport Beach, CA, USA, 1997, pp. 283–286.
- [4] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Trans. Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, May/Jun. 2000.
- [5] M. J. Zaki and K. Gouda, "Fast vertical mining using diffsets," in *Proc. 9th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, Washington, DC, USA, 2003, pp. 326–335.

参考文献 II

- [6] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Dallas, TX, USA, 2000, pp. 1–12.
- [7] C. Borgelt, "Efficient implementations of apriori and eclat," in *Proc. IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI'03)*, Melbourne, FL, USA, 2003.
- [8] B. Goethals and M. J. Zaki, "Advances in frequent itemset mining implementations: Report on FIMI'03," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 109–117, Jun. 2004.
- [9] P. Fournier-Viger, J. C.-W. Lin, A. Gomariz, T. Gueniche, A. Soltani, Z. Deng, and H. T. Lam, "The SPMF open-source data mining library version 2," in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, LNCS 9853, Riva del Garda, Italy, 2016, pp. 36–40.
- [10] University of Antwerp, "FIMI repository: Frequent itemset mining dataset repository," 2004. [Online]. Available: <https://fimi.uantwerpen.be/data/>

谢谢， 敬请指正

本报告的全部代码、数据与实验结果：

<https://github.com/timegoer/eclat-vs-apriori>